GUÍA DE USO DEL ANALIZADOR DE RED WIRESHARK

USO DEL ANALIZADOR DE RED WIRESHARK

Wireshark es el analizador de protocolos más popular en la actualidad. Se trata de un software de dominio público que está desarrollado para múltiples plataformas de sistemas operativos y múltiples interfaces físicas. Es capaz de decodificar más de 400 protocolos diferentes, desde el nivel de enlace al nivel de aplicación. El software puede obtenerse de forma gratuita de **www.wireshark.org**. Este programa es una evolución del conocido como *Ethereal*, pero su principal desarrollador pasó a trabajar para otra empresa y por problemas de registro de marcas tuvo que cambiarle el nombre.

Hay que destacar que según la versión que esté instalada en el laboratorio, algunas de las capturas de pantalla que hay en esta sección pueden ser distintas en su formato a lo que sale en el computador del laboratorio, pero la información que sacan en pantalla es la misma.

Wireshark tiene una interfaz gráfica como se muestra en la Figura 1. Cuando se utiliza sobre una interfaz Ethernet puede funcionar en modo promiscuo y capturar todas las tramas de la red, o bien en modo no promiscuo y capturar solo las tramas dirigidas a su interfaz.

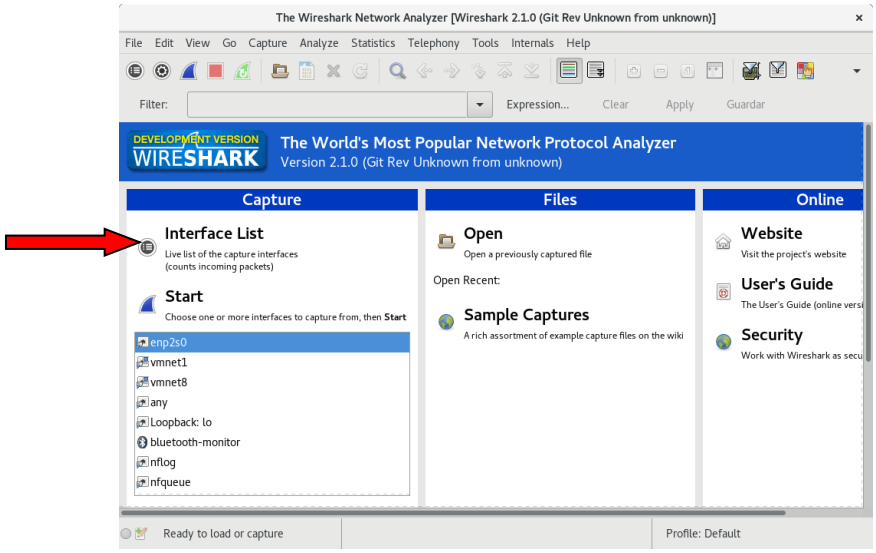


Figura 1. Interfaz gráfica del analizador de protocolos Wireshark.

La mejor manera de empezar es clicar en la '***Interface List***' que aparece en la parte superior izquierda de la ventana de la Figura 1, con lo que aparece una ventana superpuesta como la que se observa en la Figura 2.

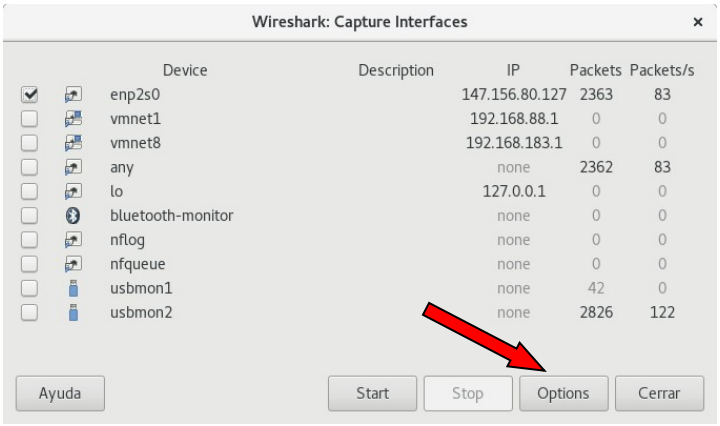


Figura 2. Menú de lista de interfaces (interfaces list)

En la lista de interfaces, como se observa en la Figura 2, se nos muestra la lista de interfaces disponibles. La columna 'IP' nos indica la dirección IP de cada interfaz, por defecto la IPv6 pero pulsando en cualquiera de las direcciones podemos obtener la IPv4. Además nos muestra la actividad de cada interfaz, indicando el número total de paquetes transmitidos/recibidos desde que se arrancó el programa, y el tráfico instantáneo en paquetes por segundo. Normalmente no tendremos problemas en identificar la interfaz Ethernet, ya que recibe el nombre **enp2s0** en la distribución fedora que está instalada en el laboratorio. Si queremos probar sockets con origen y destino en el propio computador, podemos seleccionar el interfaz lo (de loopback), cuya dirección IP es 127.0.0.1.

Si pulsamos el botón 'Start' de la interfaz Ethernet **enp2s0** empìeza de forma inmediata la captura. Sin embargo es más interesante pulsar el botón '***Options'*** de dicha interfaz como se muestra en la Figura 2, ya que esto nos permite ejercer un control previo, lo cual es muy interesante. Al hacerlo obtenemos una ventana como se muestra en la siguiente Figura:

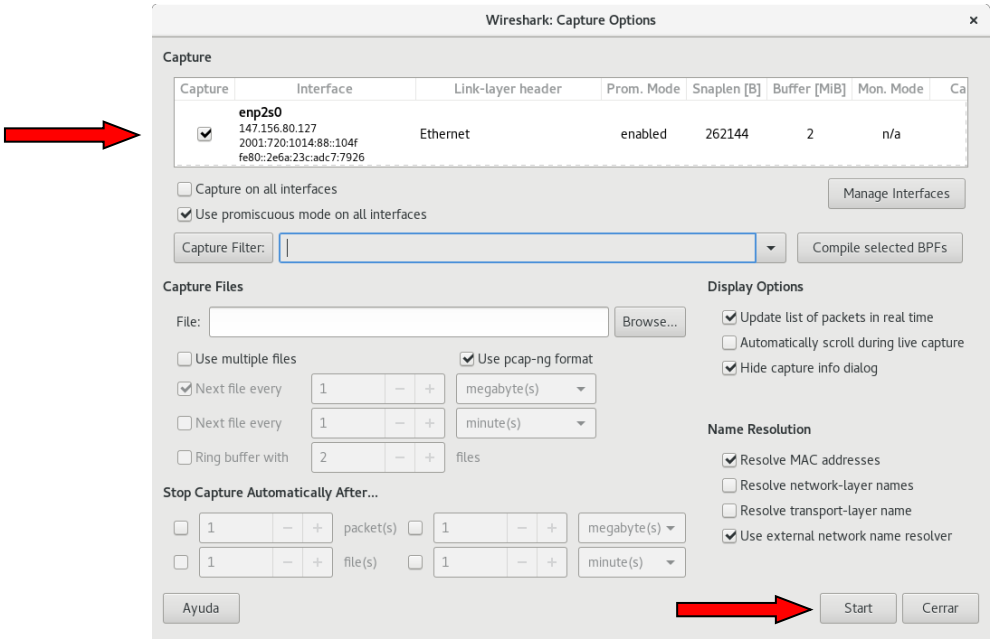


Figura 3. Menú de Capture Options para una interfaz seleccionada

En este caso en la línea que apunta la flecha se puede ver tanto la IPv6 como la IPv4 de

la interfaz en la que estamos escuchando.

Desgraciadamente no podemos explicar aquí el significado de todas las opciones que aparecen en esta ventana, por lo que solo comentaremos dos de ellas:

La pestaña '*Use promiscuous mode on all interfaces*' (marcada por defecto) nos permite indicar si queremos que la captura abarque todos los paquetes que llegan a la interfaz, o solo los que van dirigidos a ella (en cualquier caso se incluyen los paquetes *broadcast*).

El campo '*Capture Filter*' nos permite especificar un criterio de filtrado para la captura, con el fin de restringir los paquetes a aquellos que resulten interesantes para el problema o el caso que estemos siguiendo.

Cuando pulsamos el botón '***Start'*** en el menú de "*Options*" (en Figura 3), empieza inmediatamente la captura. La ventana de captura se divide en tres zonas como se muestra en la Figura 4. Puede que estas ventanas estén ocultas y haya que desplazar los separadores para verlas. En la superior se muestra la lista de tramas/paquetes capturados, a razón de una línea por paquete. En la intermedia se muestra en detalle la estructura del paquete seleccionado en la primera, y en la inferior se muestra el contenido del paquete en hexadecimal. Para analizar un paquete basta con pincharlo en la zona superior; entonces se puede seleccionar alguno de los campos que aparecen en la zona intermedia y desglosarlo en sus partes si las tiene (por ejemplo ver en detalle los campos de la cabecera IP). Al seleccionar algún campo de la zona intermedia automáticamente queda seleccionado en la inferior el contenido en hexadecimal correspondiente, con lo que es posible analizar el contenido con todo detalle. En el ejemplo mostrado en la Figura 4 se ha seleccionado en la zona superior la trama número 13; en la zona intermedia se ha seleccionado el campo **Type** de la cabecera **Ethernet** y como puede verse en la zona inferior han quedado seleccionados automáticamente los bytes 13 y 14 de la trama, que en este caso tienen el valor hexadecimal **0800**, que corresponde al protocolo IP.

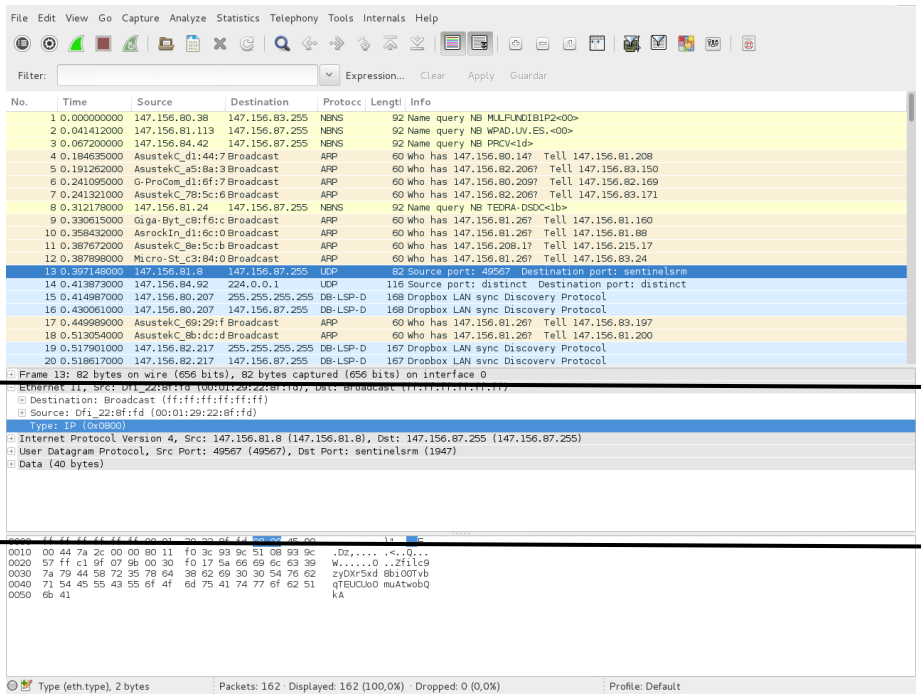


Figura 4. Pantalla de capturas de Wireshark

El analizador interpreta los protocolos desde el nivel de enlace al nivel de aplicación. La lista de protocolos que abarca para cada uno de los niveles es muy completa. Es interesante en este punto que le echéis un vistazo y vayáis pinchando por las diferentes zonas y ver, por ejemplo contenidos como los de las cabeceras de la trama Ethernet o el de las cabeceras del protocolo IP y/o IPv6.

Como la captura suele recoger una gran cantidad de paquetes, el analizador posee la opción de poder crear filtros[1] (*capture-filters*) que se aplican en el momento de hacer las capturas descartando aquellas tramas que no cumplan unas determinadas condiciones. Dichos filtros se crean desde el menú ***Capture*** y su submenú ***Options…*** escribiendo el filtro deseado dentro de la opción ***Capture Filter:***. Para poder modificar estos filtros debemos tener el proceso de capturas parado.

También existen los filtros de visualización. Estos filtros nos sirven para analizar el tráfico capturado, y son diferentes a los filtros de captura, en este caso, los filtros de visualización son más amplios y potentes, incluso son más fáciles de aplicar, y son de mucha utilidad para analizar el tráfico capturado. La principal diferencia frente a los filtros de captura es que con los filtros de captura solo capturamos el tráfico que corresponde con el filtro y en los filtros de visualización se captura todo el tráfico, pero sólo se visualiza aquel que coincida con el filtro aplicado. La introducción se realiza en la pantalla de captura en la parte superior donde se indica **Filter:**, (ver Figura 4). Para más información acerca de los filtros de visualización en Wireshark consultar el apéndice B.

Apéndice A: Filtros aplicados durante la captura por Wireshark.

*(**Nota**: estos comandos pueden sufrir modificaciones entre las diferentes versiones de Wireshark)*

En Wireshark es posible construir filtros que determinen si un determinado paquetes van a ser o no capturado. En caso de que no se utilice ningún filtro, todos los paquetes son capturados.

En Wireshark hay 2 tipos de filtros, los de captura que estamos comentando (*o capture-filters*) y los de pantalla (*display filter*). Estos últimos son diferentes a los primeros, y sirven para una vez realizada la captura, filtrar lo que se presenta en pantalla. En este apéndice sólo analizamos los primeros, los de captura. Para utilizar los filtros de pantalla, es mejor utilizar la pestaña de Expressión para crear el Filtro en la pantalla mostrada en la Figura 4.

Los filtros se construyen mediante expresiones que consisten en una o más primitivas. Las primitivas, usualmente, consisten en un identificador (nombre o número), precedidas por uno o más calificadores. Existen tres tipos diferentes de calificadores:

1 En el apéndice A y B se encuentra una explicación de los filtros y su sintaxis.

- De tipo: Identifican un nombre o dirección, sus posibles valores son *host*, *net* y *port*. Algunos ejemplos son:

Filtro	**Explicación**
`host glup.uv.es`	Paquetes cuyo origen o destino es el ordenador glup.uv.es.
`net 147.156.0.0/16`	Paquetes cuyo origen o destino es la red 147.156.0.0/16
`port 20`	Paquetes cuyo origen o destino es el puerto 20.

Si no existe ningún calificador de tipo, se asume que el tipo es *host*.

- De dirección: Identifican una dirección particular de transferencia, esto es, un origen o destino. Sus valores posibles son *src*, *dst*, *src or dst* y *src and dst*. Por ejemplo:

Filtro	**Explicación**
`src glup.uv.es`	Paquetes cuyo origen es el ordenador glup.uv.es.
`dst net 147.156.0.0/16`	Paquetes destino es la red 147.156.0.0/16
`src or dst port ftp-data`	Paquetes cuyo origen o destino es el puerto 20 (ftp-data).

Si no se indica ningún calificador de dirección, se toma el calificador de dirección por defecto (*src or dst*).

- De protocolo: Identifican un protocolo particular. Sus valores posibles son *ether*, *fddi*, *tr*, *ip*, *ip6*, *arp*, *rarp*, *decnet*, *tcp* y *udp*. Por ejemplo[2]:

Filtro	**Explicación**
`ether src glup.uv.es`	Paquetes ethernet cuyo origen es el ordenador glup.uv.es.
`arp net 147.156`	Paquetes de protocolo ARP con origen o destino en la red 147.156.
`tcp port 21`	Paquetes cuyo protocolo es TCP y origen o destino en el puerto 21.

Si no se especifica ningún protocolo, todos los protocolos que sean consistentes con la identificación de tipo son capturados.

Como puede verse por la explicación de los diferentes tipos de calificadores, siempre están presentes, aunque sea por defecto, los tres tipos de calificadores. Así, la expresión:

```
ip 147.156.222.65
```

Es equivalente a:

```
ip src or dst host 147.156.222.65
```

2 Si en algún momento se desea especificar una dirección ethernet en el filtro, está debe especificarse como XX:XX:XX:XX:XX:XX

Pueden construirse filtros más complejos mediante la combinación de primitivas mediante la utilización de paréntesis y/o las palabras *and*, *or* y *not*, siendo la prioridad de *not* mayor que la de *and* y *or*, cuya prioridad entre si es igual. Así, por ejemplo:

```
host glup.uv.es and not port ftp
```

Indica que se capturen todos los paquetes cuyo origen o destino es glup.uv.es excepto aquellos cuyo puerto de origen o destino es el de ftp (puerto 21).

Un listado de las primitivas más utilizadas se encuentra en la siguiente tabla:

Primitiva	**Descripción**
`dst host <ordenador>`	Verdad si el campo destino del paquete es el <*ordenador*>.
`src host <ordenador>`	Verdad si el campo origen del paquete es el <*ordenador*>.
`host <ordenador>`	Verdad si el campo origen o destino del paquete es el <*ordenador*>.
`Ether dst <ordenador>`	Verdad si la dirección ethernet de destino es el <*ordenador*>.
`Ether src <ordenador>`	Verdad si la dirección ethernet de origen es el <*ordenador*>.
`Ether host <ordenador>`	Verdad si la dirección ethernet de origen o destino es el <*ordenador*>.
`dst net <red>`	Verdad si la dirección de destino del paquete corresponde a una dirección de la <*red*>.
`src net <red>`	Verdad si la dirección de origen del paquete corresponde a una dirección de la <*red*>.
`net <red>`	Verdad si la dirección de origen o destino del paquete corresponde a una dirección de la <*red*>.
`dst net <red> mask <mascara>`	Verdad si la dirección de destino del paquete corresponde a una dirección de la <*red*> de máscara <*mascara*>.
`src net <red> mask <mascara>`	Verdad si la dirección de origen del paquete corresponde a una dirección de la <*red*> de máscara <*mascara*>.
`net <red> mask <mascara>`	Verdad si las dirección de origen o destino del paquete corresponde a una dirección de la <*red*> de máscara <*mascara*>.
`dst net <red>/<longitud>`	Verdad si la dirección de destino del paquete corresponde a una dirección de la <*red*> cuya máscara se indica por <*longitud*>.
`src net <red>/<longitud>`	Verdad si la dirección de origen del paquete corresponde a una dirección de la <*red*> cuya máscara se indica por <*longitud*>.
`net <red>/<longitud>`	Verdad si las dirección de origen o destino del paquete corresponde a una dirección de la <*red*> cuya máscara se indica por <*longitud*>.
`dst port <puerto>`	Verdad si el paquete tiene como destino el puerto dado por <*puerto*>[3].

Primitiva	**Descripción**
`src port <puerto>`	Verdad si el paquete tiene como origen el puerto dado por <*puerto*>.
`port <puerto>`	Verdad si el paquete tiene como origen o destino el puerto dado por <*puerto*>.
`less <longitud>`	Verdad si el paquete tiene una longitud menor o igual que <*longitud*>.
`greater <longitud>`	Verdad si el paquete tiene una longitud mayor o igual que <*longitud*>.

Además de las expresiones anteriores, existen expresiones del tipo:

```
<expresión 1> <operador> <expresión 2>
```

Donde <*operador*> es uno de los operadores de comparación <, >, <=, >=, = y *!=* (menor, mayor, menor o igual, mayor o igual, igual y distinto).

Por su parte, <*expresión 1*> y <*expresión 2*> son expresiones aritméticas compuestas por constantes enteras expresadas con la sintaxis de C/C++ y los operadores +, -, *, /, *&* y | (suma, resta, producto, división, and lógico y or lógico). Además, existe una expresión especial que permite acceder a unos datos concretos del paquete. Esta expresión que permite el acceso a los datos de un paquete tiene la sintaxis:

```
protocolo [ desplazamiento : tamaño]
```

Donde *protocolo* es uno de los protocolos validos (*ether*, *fddi*, *tr*, *ip*, *arp*, *rarp*, *tcp*, *udp*, *icmp* o *ip6*), *desplazamiento* es el desplazamiento, en bytes, desde el comienzo de los datos del protocolo especificado (el primer byte es el byte cero), y *tamaño* son los bytes a analizar, tomando por defecto el valor de 1 byte si no se especifica ningún valor. Por ejemplo:

```
ip[0 : 1] & 0x0F != 5
```

Indica todos los paquetes que contienen opciones IP (campo longitud de la cabecera con valor distinto de 5). Esta expresión también puede escribirse como:

```
ip[0] & 0x0F != 5
```

Teniendo en cuenta que si no se especifica el tamaño se toma por defecto 1. Por otra parte, el siguiente ejemplo:

```
ip[6 : 2] & 0x1FFF == 0
```

Indica solo datagramas no fragmentados o el primer fragmento de los datagramas fragmentados (campo desplazamiento del fragmento igual a cero).

Otros comandos de IPv6: Capture Filter (https://wiki.wireshark.org/IPv6)

3 Esta expresión y las dos siguientes pueden ir precedidas de tcp ó udp, para indicar que solo se desea el puerto correspondiente al protocolo TCP ó UDP respectivamente.

Captura de sólo tráfico IPv6:

```
ip6
```

Captura de tráfico IPv6 del host fe80::1:

```
host fe80::1
```

Captura tráfico de IPv6 en túnel IPv6-over-IPv4

```
ip proto 41
```

Captura de sólo tráfico IPv6 nativo:

```
ip6 and not ip proto 41
```

Apéndice B: Filtros aplicados en la visualización de tráfico en Wireshark.

Los filtros de visualización (Display Filter) establecen los criterios de filtro sobre los paquetes capturados y que estamos visualizando en la pantalla principal de Wireshark.

Como puede verse en la figura 4, el área para escribir un filtro de captura es la ubicada debajo de la barra de herramientas. Wireshark nos proporciona una retroalimentación visual para realizar los filtros de visualización. Si la expresión de filtrado que escribimos es correcta, el área se colorea en verde mientras si la expresión es incorrecta, el área se colorea en rojo, y al tratar de aplicarlo obtendremos un mensaje de error.

Con los filtros de visualización veremos solo los paquetes que cumplan los requisitos indicados en el filtro. Si no establecemos ninguno, Wireshark mostrará todo el tráfico capturado y lo presentará en la pantalla principal.

Estos filtros son mucho más potentes que los de captura y mucho más fáciles de utilizar. La lista es inmensa como podemos ver en https://www.wireshark.org/docs/dfref/

Podemos combinar los filtros de la siguiente forma:

Negación: `!` ó `not`
Concatenación: `&&` ó `and`
Alternancia: `||` ó `or`

Los operadores que podemos utilizar para construir expresiones de filtro son los siguientes:

Mayor que: `>` ó `gt`
Menor que: `<` ó `lt`
Mayor o igual que: `>=` ó `ge`
Menor o igual que: `<=` ó `le`
Igual que: `==` ó `eq`
No es igual a: `!=` ó `ne`

Contains: busca la cadena *contains*
busca la cadena "google.es" en los paquetes tcp: `tcp contains "www.google.es"`

Creación de filtros

Los filtros de visualización ofrecen una gran cantidad de posibilidades a la hora de mostrar unos paquetes u otros. Por suerte, Wireshark posee un buscador para facilitarnos la tarea. Para familiarizarnos un poco con este tipo de filtros vamos a ver unos cuantos ejemplos.

Filtrado por protocolo

Para visualizar los paquetes **arp**

```
arp
```

Para visualizar paquetes **icmp** como el *ping*.
```
icmp
```

Para visualizar los paquetes **udp**
```
udp
```

Para visualizar los paquetes **tcp**
```
tcp
```

Filtrar por direcciones ip

Para visualizar todos los paquetes ip
```
ip
```

Para visualizar paquetes cuya ip de origen o destino es 192.168.100.1
```
ip.addr == 192.168.100.1
```

Para visualizar paquetes cuya ip de origen es 192.168.100.1
```
ip.src == 192.168.100.1
```

Para visualizar paquetes cuya ip de destino es 192.168.100.1
```
ip.dest == 192.168.100.1
```

Podemos filtrar por puerto

Para visualizar paquetes tcp cuyo puerto de origen o destino es el 80
```
tcp.port == 80
```

Para visualizar paquetes tcp cuyo puerto de origen es el 23
```
tcp.srcport == 23
```

Para visualizar paquetes tcp cuyo puerto de destino es el 23
```
tcp.dstport == 23
```